Brain-to-Text: A Two-Stage Decoding Pipeline with Conformer and LLM

Hou Yiwei
matriculation number: 13019370
yiwei.hou@campus.lmu.de

Wang Zhe
matriculation number: 13058209
Zhe.Wang@campus.lmu.de

Contents

1 Introduction **2**
1.1 The Acoustic Module (Neural-to-Phoneme) . . . 2
1.2 The Language Module (Phoneme-to-Text) . . . 2

2 Dataset and Preprocessing **3**
2.1 Neural Dataset and Acoustic Preprocessing . . . 3
2.2 Phoneme Vocabulary . . . 3
2.3 Synthetic Data Generation and LM Augmentation . . . 3

3 Experimental Setup and Training Details **4**
3.1 Acoustic Model . . . 4
3.1.1 EEG Conformer . . . 4
3.1.2 DBConformer . . . 5
3.2 Language Model Fine-Tuning and Prompting . . . 10
3.2.1 Parameter-Efficient LLM Fine-Tuning (Qwen 2.5 7B) . . . 10
3.2.2 Encoder-Decoder Models (BART & ByT5) . . . 10
3.3 Training Time and Hardware Used . . . 11

4 Results and Evaluation **11**
4.1 Acoustic Model . . . 11
4.2 Language Model . . . 12
4.3 Error Examination . . . 13

5 Uncertainty Estimation and Model Calibration **14**
5.1 Acoustic Model Uncertainty . . . 14
5.2 Language Model Uncertainty . . . 15
5.2.1 BART . . . 15
5.2.2 Qwen 2.5-7B . . . 16
5.3 Comparative Analysis . . . 17

6 Conclusion and Future Work **18**

1 Introduction

This report introduces the complete pipeline for translating raw neural signals into readable English text. The system employs a two-stage architecture: an Acoustic Model (AM) that decodes neural electrophysiological data into an intermediate sequence of phonemes, followed by a Language Model (LM) that corrects and translates these noisy phoneme sequences into grammatically correct text.

1.1 The Acoustic Module (Neural-to-Phoneme)

The first stage of the pipeline focuses on extracting phonetic information directly from the neural signal data.

Feature Extraction & Downsampling: The raw neural signals (e.g., 512 channels) are first normalized and processed through a convolutional stem. This step includes spatiotemporal convolutions and average pooling to reduce the temporal length of the sequence, yielding a token rate suitable for phoneme boundaries.

Acoustic Modules: We explored two distinct Conformer (Convolutional Transformer) architectures to process these sequences:

- **EEG Conformer** [1]: First uses a temporal layer to extract local temporal features. Then it is followed by a Spatial Convolution to learn the correlations between electrode channels. A pooling layer is used to suppress noise and slices the data into temporal segments (tokens). A standard 6-layer Transformer encoder to capture global temporal dependencies was used before two fully-connected layers followed by a Softmax function to output category probabilities.
- **DBConformer (Dual-Branch)** [2]: Instead of stacking convolutions sequentially like the EEG Conformer, it parallels these computations through a T-Conformer for hierarchical temporal dynamics and an S-Conformer for global spatial patterns. This design avoids "temporal bottlenecks" while a lightweight channel attention module adaptively re-weights specific EEG channels before the branch outputs are concatenated for classification.

CTC Decoding: Both architectures output to a Connectionist Temporal Classification (CTC) head. The CTC loss function is crucial because it aligns the unsegmented neural sequence with discrete phoneme targets (40 standard English phonemes plus a "blank" token). During inference, CTC decoding algorithms collapse repeated tokens and remove blanks, yielding the raw predicted phoneme sequence.

1.2 The Language Module (Phoneme-to-Text)

The Language Model Pipeline acts as an error-correcting decoder that translates noisy, predicted phoneme sequences from the Acoustic Modules into readable English text. To make the LM robust to acoustic errors, it employs a data augmentation strategy that injects targeted substitutions, insertions, and deletions based on an empirical phoneme confusion matrix (creating a training mix of 20% ground truth, 60% synthetic noise, and 20% real acoustic predictions). The pipeline explores three distinct decoding approaches:

- **Parameter-Efficient LLM Fine-Tuning:** Adapting a localized instruction-tuned

Source code available on GitHub: https://github2com/hyw-pz/applied-dl/tree/main

LLM (Qwen 2.5 7B) using 4-bit quantization and QLoRA to learn the phoneme-to-text mapping.

- **Seq2Seq Fine-Tuning:** Treating the task as machine translation by fine-tuning encoder-decoder architectures like BART (Base/Large) and ByT5.

Finally, the models are evaluated using Word Error Rate (WER), BLEU, and ROUGE-L metrics to measure their spelling and grammatical accuracy.

2 Dataset and Preprocessing

2.1 Neural Dataset and Acoustic Preprocessing

The primary dataset, derived from the Brain-to-Text challenge and originally detailed by Card et al. [3], consists of continuous neural electrophysiological signals collected from a single research participant over 45 sessions spanning 20 months. Highlighting the data efficiency of the neuroprosthesis pipeline, the initial acoustic model was calibrated with approximately 1.5 hours of neural recordings, while subsequent model updates relied on daily rapid calibration sessions of only 10–20 minutes. During the recording sessions, the subject was instructed to attempt speaking various sentences—employing either attempted vocalized or attempted silent speaking strategies—drawn from multiple corpuses such as Switchboard and Harvard sentences. The neural activity was recorded via 256 intracortical microelectrodes implanted in the speech motor cortex, yielding 512 distinct neural feature channels (comprising both root-mean-square threshold crossings and spike band power for each electrode) binned at a 20 ms resolution. This raw data is stored in HDF5 format, and the overall evaluation of the system was conducted utilizing a validation set of 1,378 trials.

To prepare these signals for the Convolutional Transformers, sequences are truncated or dynamically padded to a maximum of 1,500 time steps to maintain computational efficiency (capturing 98% of the overall sequence lengths). The raw features are Z-score normalized for a stable, zero-mean distribution, then reshaped into 4D tensors (Batch, 1, 512, Time) for convolutional layer compatibility.

2.2 Phoneme Vocabulary

The Acoustic Model decodes the neural signals into a vocabulary of 41 distinct classes. This includes 40 standard English ARPAbet phonemes (covering all necessary consonants and vowels) and 1 Connectionist Temporal Classification (CTC) "blank" token, which is essential for aligning the unsegmented neural data.

2.3 Synthetic Data Generation and LM Augmentation

Because the Acoustic Model's intermediate outputs naturally contain misclassified or missing phonemes, we employ a data augmentation strategy to train the error-correcting Language Model (LM). By analyzing acoustic validation errors, we mapped an empirical phoneme confusion matrix to inject realistic noise. The final LM training corpus utilizes a robust 20/60/20 data split:

- **20% Ground Truth:** Clean, perfect reference sequences.

- **60% Synthetic Noisy Data:** Ground truth sequences artificially injected with targeted substitutions (similar sounds), random substitutions, insertions, and deletions based on the confusion matrix (20%, 50%, 20%, 10%). The percentages are obtained by observing the real mistake types.
- **20% Real Predicted Data:** Actual noisy outputs generated by the DBConformer.

This mix prevents the LM from overfitting to clean text and teaches it to reconstruct intended sentences from realistic neural decoding errors. The details of the targeted substitutions can be found in Appendix 6.

3 Experimental Setup and Training Details

This section details the architectural adjustments and hyperparameters employed in both the AM and LM models.

3.1 Acoustic Model

The Acoustic Model (AM) acts as the foundational feature extractor, decoding continuous neural signals into intermediate phoneme probabilities. Our architecture is based on the DBConformer (Dual-Branch Convolutional Transformer), which improves upon hybrid models like the EEG Conformer. We mainly focused on the DBConformer in this experiment as it provides better training efficiency.

3.1.1 EEG Conformer

In the EEG Conformer, features are processed sequentially: the model first applies temporal and spatial convolutions to extract local features, slices them with average pooling, and feeds them into a self-attention module followed by a classifier.

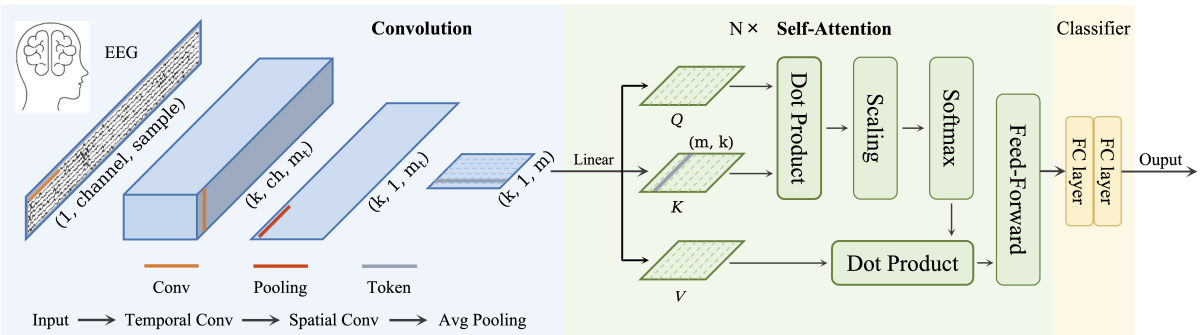

Figure 1: Architecture of the original EEG Conformer (Source: EEGConformer: Convolutional Transformer for EEG Decoding and Visualization; Song et al. (2023))

The adopted architecture and hyperparameters are summarized in Tables 1 and 2.

Table 1: EEG Conformer Architecture

Module	Layer	Kernels	Kernel Size	Output Shape	Options
Patch Embedding	Temporal Conv2D	D	$(1, K_t)$	(B, D, C, T)	Temporal feature extraction (padding applied)
	Spatial Conv2D	D	$(C, 1)$	$(B, D, 1, T)$	Depthwise spatial filtering (groups $= D$)
	BatchNorm2D & ELU	—	—	$(B, D, 1, T)$	Normalization and non-linearity
	AvgPool2D	—	$(1, W)$	$(B, D, 1, P)$	Temporal downsampling (Stride $= (1, S)$)
	Dropout	—	—	$(B, D, 1, P)$	$p = 0.5$
	Projection (Conv2D)	D	$(1, 1)$	$(B, D, 1, P)$	Channel projection to target embedding
	Rearrange	—	—	(B, P, D)	Flatten to sequence tokens
Transformer Encoder	Positional Encoding	—	—	(B, P, D)	Sinusoidal encoding & Dropout ($p =$ drop_p)
	Transformer Block	—	—	(B, P, D)	L layers, H heads (Pre-LN, FlashAttention)
	LayerNorm	—	—	(B, P, D)	Final encoder normalization
CTC Head	Linear & GELU	—	—	(B, P, FFN)	Intermediate feature projection
	Dropout	—	—	(B, P, FFN)	$p =$ drop_p
	Linear & Permute	—	—	(P, B, N_c)	Project to N_c classes and log_softmax

Table 2: Hyperparameters for EEG Conformer

Parameter	Symbol	Value
Batch Size	B	28
Number of Electrodes (Channels)	C	512
Number of Time Points	T	1500
Temporal Kernel Size	K_t	10
Pooling Window Size	W	15
Pooling Stride	S	8
Embedding Dimension	D	128
Transformer Layers (Depth)	L	6
Attention Heads	H	8
Feed Forward Hidden Size	FFN	256
Number of Classes	N_c	41

The innovations we adopted for the EEG Conformer include added positional encoding, masked threading, and modification of the final classification layer to adopt the CTC Head. These are detailed in Table 3 as DBConformer also adopted these techniques.

3.1.2 DBConformer

Instead of the serial constraint the EEG Conformer has, the DBConformer explicitly decouples feature extraction into two parallel branches:

- **T-Conformer:** A temporal branch that integrates depthwise temporal convolutions with multi-head self-attention to capture long-range temporal dependencies.
- **S-Conformer:** A spatial branch that utilizes channel-wise convolutions and self-attention to model inter-channel spatial interactions.

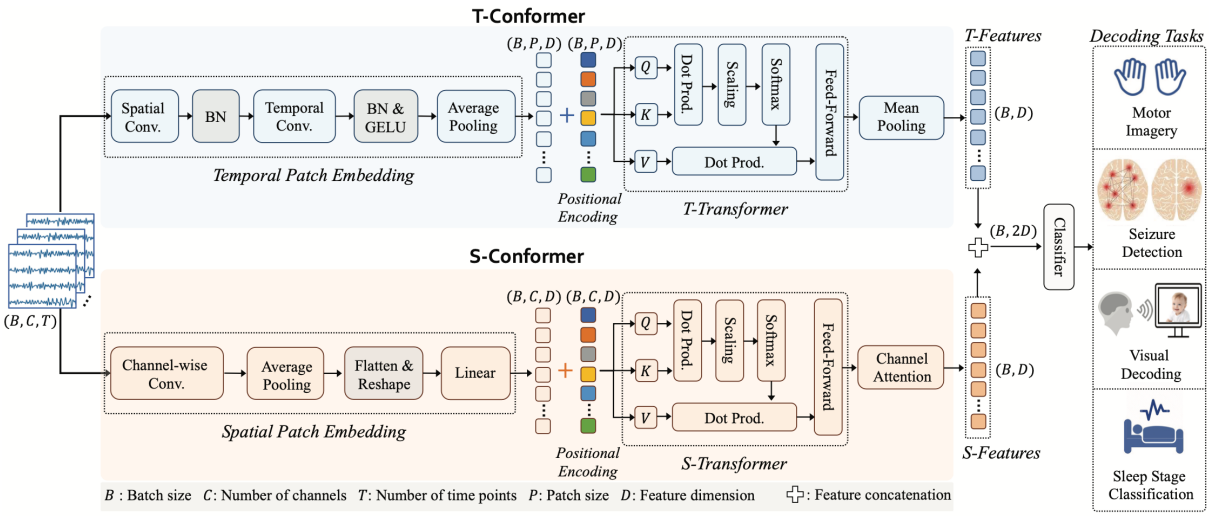

Figure 2: Architecture of the original DBConformer (Source: DBConformer: Dual-Branch Convolutional Transformer for EEG Decoding; Wang et al. (2025))

To adapt the DBConformer from its original static classification tasks to continuous sequence-to-sequence phoneme decoding, we introduced several fundamental architectural innovations, summarized in Table 3.

Table 3: DBConformer Improvements

Feature / Module	**Original Paper**	**Our Code Innovation**	**Benefit**
Adaptation for Continuous Sequence Decoding	Applies mean pooling to both branches, concatenates into a 1D vector, and uses an MLP for static classification.	Bypasses global mean pooling; routes the unpooled 2D temporal sequence from the T-Conformer directly to a CTC head.	Processes variable-length streams and aligns them asynchronously to continuous speech tokens over time.
			Continued on next page

Table 3 – continued

Feature / Module	**Original Paper**	**Our Code Innovation**	**Benefit**
Spatial Branch Bypassing	Outputs from T-Conformer and S-Conformer are concatenated before classification.	Bypasses S-Conformer output and omits Channel Attention; routes only T-Conformer output to the CTC head.	Prevents time-agnostic spatial data from disrupting the strict temporal alignment needed for rapid phonetic detection.
Mask Threading & Flash Attention	Uses standard self-attention mechanisms.	Uses scaled dot-product attention and explicitly threads a boolean padding mask through all T-Transformer layers.	Improves memory efficiency and prevents zero-padded regions in dynamic batches from corrupting attention weights.
Deep Position-wise CTC Classifier	Uses a 3-layer MLP dimension reduction for dataset-level classification.	Uses a deeper, position-wise FFN (LayerNorm → Linear → GELU → Dropout → Linear) with 512-dim states, formatted for PyTorch CTC.	Adds non-linear capacity to translate contextual embeddings into 41 frame-by-frame phoneme probabilities.
Overlapping Patches (Pooling/Stride)	Uses non-overlapping average pooling (patches equal time divided by window).	Separates the pooling kernel (15) and stride (8) to create overlapping temporal patches.	Retains higher temporal resolution and smooths contextual transitions to accurately track sequential tokens.
Dropout & Pooling Order	Applies Dropout immediately after GELU, before Average Pooling.	Reverses the order: applies Average Pooling first, followed by Dropout on the downsampled feature map.	Increases computational efficiency and provides more stable regularization by dropping compacted, higher-level features.

Continued on next page

Table 3 – continued

Feature / Module	**Original Paper**	**Our Code Innovation**	**Benefit**
ELU Activation in Spatial Branch	Uses channel-wise Conv1D immediately followed by Average Pooling.	Introduces an ELU activation directly after the channel-wise Conv1D, before spatial downsampling.	Adds early non-linearity to the spatial pipeline, improving the modeling of complex inter-channel relationships.

Hyperparameter Summaries for the DBConformer Module

The following tables outline the specific kernel configurations, strides, and dimension transformations for the DBConformer as implemented in our codebase.

Table 4: DBConformer Architecture

Branch	**Module**	**Layer**	**Kernels**	**Size**	**Params**	**Output Shape**	**Options**
T-Conformer	Temporal Patch Embedding	Separable Conv1D	F	$(1,)$	$C * F$	(B, F, T)	Spatial filtering
		BatchNorm1D	—	—	$2F$	(B, F, T)	Normalization
		Temporal Conv1D	F	$(K,)$	$F * K$	(B, F, T)	Temporal depth-wise Conv.
		BatchNorm1D & GELU	—	—	$2F$	(B, F, T)	Normalization & non-linearity
		AvgPool1D	—	$(1, W)$	0	(B, F, P)	Stride $= S$
		Dropout	—	—	0	(B, F, P)	$p = 0.5$ (after pooling)
		Permute	—	—	0	(B, P, F)	Patch tokens along time
T-Transformer	Attention	Positional Encoding	—	—	$P * D$	(B, P, D)	Learnable temporal enc.
		Transformer Encoder	—	—	$\sim$	(B, P, D)	L_t layers, H_t heads

Continued on next page

Table 4 – continued from previous page

Branch	Module	Layer	Kernels	Size	Params	Output Shape	Options
S-Conformer (calc., discarded)	Spatial Patch Embedding	Channel-wise Conv1D	16	25	—	$(BC, 16, T')$	Stride = 5
		ELU	—	—	0	$(BC, 16, T')$	Non-linearity
		Adaptive Avg-Pool1D	—	—	0	$(BC, 16, 1)$	Channel downsampling
		Flatten	—	—	0	$(BC, 16)$	Reshape
		Linear	16	—	$16D$	(BC, D)	Project to embedding
		Reshape	—	—	0	(B, C, D)	Channel-token prep.
S-Transformer	Attention	Positional Encoding	—	—	$C * D$	(B, C, D)	Learnable spatial enc.
		Transformer Encoder	—	—	$\sim$	(B, C, D)	L_s layers, H_s heads
CTC Head	Classifier	LayerNorm	—	—	$2D$	(B, P, D)	Normalization
		Linear & GELU	—	—	$D * \text{FFN}$	(B, P, FFN)	Intermediate projection
		Dropout	—	—	0	(B, P, FFN)	$p = 0.3$
		Linear & Permute	—	—	$\text{FFN} * N_c$	(P, B, N_c)	Project to classes

Table 5: Hyperparameters for DBConformer

Parameter	Symbol	Value
Batch Size	B	28
Number of Channels	C	512
Number of Time Points	T	1500
Number of Filters	F	512
Temporal Kernel Size	K	15
Pooling Window Size	W	15
Pooling Stride	S	8
Embedding Dimension	D	512
T-Transformer Encoder Layers	L_t	6
S-Transformer Encoder Layers	L_s	6
T-Transformer Encoder Heads	H_t	8
S-Transformer Encoder Heads	H_s	8
Number of Classes	N_c	41
Feed Forward Hidden Size	FFN	1024 ($= 2D$)

3.2 Language Model Fine-Tuning and Prompting

The second stage of the pipeline translates the noisy ARPAbet phoneme sequences into English text. We explored three separate strategies with different training sets. All of the following modules used BEAM = 4 in decoding.

3.2.1 Parameter-Efficient LLM Fine-Tuning (Qwen 2.5 7B)

To leverage a state-of-the-art causal LLM without exceeding GPU memory constraints, we fine-tuned the `Qwen/Qwen2.5-7B-Instruct` model using QLoRA (Quantized Low-Rank Adaptation).

Quantization: The base weights were frozen and loaded in 4-bit precision using the Normalized Float 4 (NF4) data type with double quantization (`bnb_4bit_use_double_quant=True`) and bfloat16 compute data types.

LoRA Setup: Adapters were applied to all linear attention and feed-forward layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) with a rank $r = 16$, alpha = 32, and dropout = 0.1.

System prompt: "You are an expert speech decoding system. Translate the noisy ARPAbet phonemes into English text. CRITICAL CONSTRAINTS: You must ONLY output valid English dictionary words. Output ONLY the final text."

Training: The model was trained using the `paged_adamw_8bit` optimizer and a cosine learning rate scheduler. Data was strictly formatted into Qwen's native ChatML template format.

3.2.2 Encoder-Decoder Models (BART & ByT5)

We framed phoneme-to-text as a machine translation task using HuggingFace's `Seq2SeqTrainer`. BART and ByT5 are transformer-based sequence-to-sequence models with fundamentally different tokenization strategies. While BART uses a subword tokenizer with a fixed vocabulary, ByT5 processes raw bytes or characters directly. This byte-level approach makes ByT5 more robust to misspellings and noise without out-of-vocabulary errors, but it results in significantly longer sequence lengths compared to BART.

- **BART-base:** Trained for 50 epochs with a per-device batch size of 80. Initial learning rate of 1×10^{-5} and weight decay of 0.01. Training uses 16-bit floating-point precision (fp16). An early stopping callback with patience of 5 epochs is implemented based on WER.
- **BART-large:** Trained for 50 epochs with a batch size of 80 (or 60 for synthetic data variants). Initial learning rate of 2×10^{-7}, warmup ratio of 0.1, and weight decay between 0.03 and 0.05. A label smoothing factor of 0.1 is applied, and gradients are clipped at a maximum norm of 1.0. Configured with bfloat16 precision (bf16), enabled weight tying, and a disabled forced BOS token.
- **ByT5-Small:** Because ByT5 operates on byte-level characters, the maximum input/target lengths were expanded to 512. Trained with a learning rate of 5×10^{-5} and gradient accumulation steps set to 4.

3.3 Training Time and Hardware Used

Table 6: Training Time and Hardware Summary

Model	Variant	Hardware	Time	Epochs
EEGConformer	emb = 128	L4	10h	120
DBConformer	emb = 128	T4	0.5h	100
DBConformer	emb = 256	L4	0.5h	100
DBConformer	emb = 512	L4	∼1h	100
BART-base	50% GT + 50% predicted	L4	0.5h	50
BART-large	50% GT + 50% predicted	L4	1h	50
Qwen2.5-7B (QLoRA)	20% GT + 60% synth + 20% predicted	L4	7h	3

4 Results and Evaluation

4.1 Acoustic Model

The Acoustic Model is evaluated by examining the distribution of Phoneme Error Rate (PER). A summary is provided in Table 7.

Table 7: Summary Results for the Acoustic Model

Model	Embedding Size	Parameter Count	Avg. PER	Median PER
EEG Conformer	128	1.3M	0.272	0.258
DBConformer	128	2.6M	0.193	0.175
DBConformer	256	9.8M	0.179	0.154
DBConformer	512	38.5M	0.124	0.090

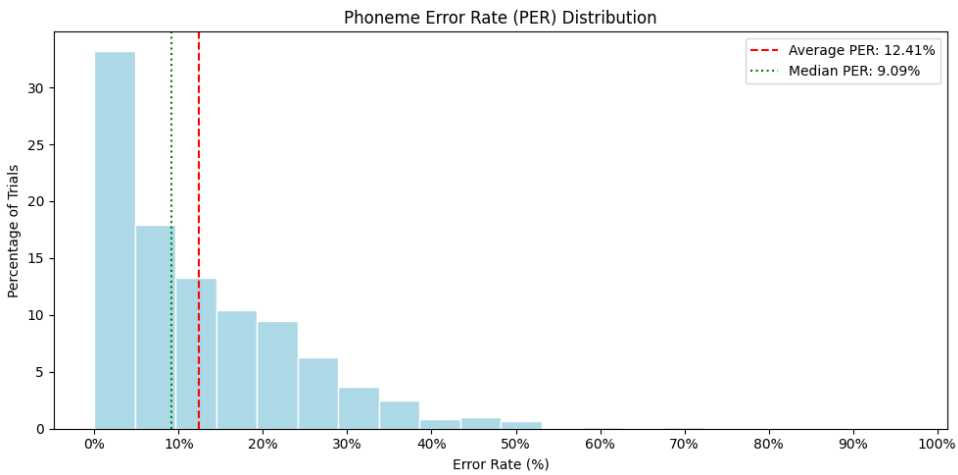

Figure 3: Distribution of PER for DBConformer (embedding size 512)

The histogram of PER for the best-performing model (DBConformer 512) reveals a highly accurate acoustic model, with over 30% of trials falling in the 0–5% error range. The

average of 12.41% is skewed by a small number of higher-error outliers. This right-skewed distribution confirms consistent precision across the vast majority of evaluated brain signal sequences.

4.2 Language Model

The Language Model results are summarized in Table 8. GT stands for Ground Truth, pred stands for predicted training set.

Table 8: Summary Results for Language Model Variants

Model	Training Data	Avg. WER	Med. WER	BLEU	R-1	R-2	R-L
Qwen2.5-7B (QLoRA)	50% GT + 50% pred.	0.243	0.17	0.595	0.783	0.658	0.783
Qwen2.5-7B (QLoRA)	20% GT + 60% synth + 20% pred.	**0.224**	**0.12**	**0.645**	**0.804**	**0.693**	**0.804**
BART-base	50% GT + 50% pred.	0.248	0.20	0.591	0.760	0.629	0.759
BART-base (synth)	25% GT + 25% synth + 50% pred.	0.273	0.20	0.582	0.744	0.622	0.744
BART-large	50% GT + 50% pred.	0.249	0.20	0.588	0.758	0.626	0.757
BART-large (synth)	20% GT + 60% synth + 20% pred.	0.275	0.20	0.571	0.742	0.613	0.742
ByT5-small	50% GT + 50% pred.	0.407	0.40	0.370	0.655	0.482	0.654

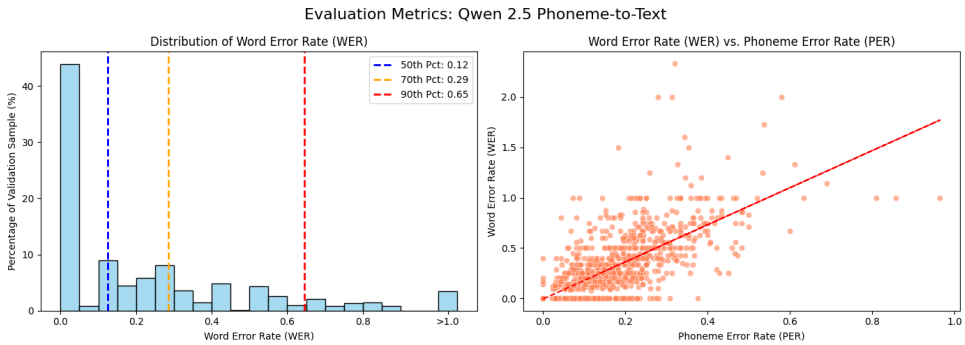

Figure 4: Evaluation metrics for the Qwen 2.5 model. The figure displays the overall WER distribution (left) and the relationship between PER and WER (right) on the validation set.

The fine-tuned Qwen 2.5-7B module demonstrates strong ability in translating phonemes to text and error correction. The WER distribution is highly right-skewed, with over 40% of samples having 0 error and a median WER of 0.12. Linguistic quality remains high, evidenced by a BLEU score of 0.645 and a ROUGE-L of 0.804. The error correction capability is confirmed by numerous observations with PER > 0 accompanied by WER $= 0$.

4.3 Error Examination

While the Qwen 2.5-7B module demonstrates strong error correction capabilities, analyzing instances where the Word Error Rate (WER) is greater than zero reveals four distinct categories of language model failures. These categories highlight the trade-offs of using an instruction-tuned Large Language Model for phonetic translation.

1. Format and Contraction Retention In some cases, the Acoustic Model (AM) correctly predicts all phonemes, but the LM fails to perfectly match the ground truth due to formatting biases. The LM often outputs semantically equivalent but strictly mismatched text by introducing or expanding contractions (e.g., hallucinating "cannot" as "can't") or altering word spacing (e.g., "any more" to "anymore").

2. Semantic Divergence via Phonetic Proximity When the acoustic signal produces valid but incorrect phonemes, the LM faithfully translates these erroneous sounds into grammatically correct English, resulting in a completely divergent meaning. For example, when the phonemes for prime time football game were degraded to P AE K | T AY M | S T AH B AH L | K IY M, the model successfully decoded the noisy sounds into English but output the nonsensical phrase pack time stabilizer.

3. Contextual Overcompensation (Semantic Flipping) The LM heavily relies on its internal language priors to fill in missing gaps when phonemes are ambiguous. This can cause the model to insert highly plausible but factually incorrect words, sometimes even producing antonyms. For instance, when translating My husband travels frequently, the AM dropped the initial phonemes for "frequently" (F R IY...), outputting R IY K W AH N L IY. The LM recognized the context of a modifying adverb for "travels" but overcompensated, outputting the opposing phrase My husband travels rarely.

4. Catastrophic Hallucination on High-Noise Inputs When the Phoneme Error Rate (PER) is excessively high, the original acoustic meaning is lost. Because the LM is instruction-tuned to output valid English sentences, it completely abandons the reference and hallucinates a coherent sentence loosely built on the remaining phonetic fragments. For example, when the sentence The town is just over the hill here was garbled into W IY | M EH | SH R EH JH | ER ZH ER..., the LM forced grammatical coherence out of the noise and predicted We may regret our decision later.

Table 9: Error Examination Examples

Error Category	**Reference**	**Prediction**	**WER**	**PER**
Format / Contraction	We cannot go there without you.	We can't go there without you.	0.167	0
Format / Contraction	They don't make them anymore.	They don't make them any more.	0.400	0
Phonetic Proximity	It was a prime time football game.	It was a pack time stabilizer.	0.428	0.310
Phonetic Proximity	It is impressive in more than just quantity.	It is expressive in more than just quantity.	0.125	0.125
Contextual Overcompensation	My husband travels frequently.	My husband travels rarely.	0.250	0.066
Catastrophic Hallucination	The town is just over the hill here.	We may regret our decision later.	1.000	0.633

5 Uncertainty Estimation and Model Calibration

5.1 Acoustic Model Uncertainty

To evaluate the reliability of the acoustic model's phoneme predictions, we compute a sequence-level uncertainty score based on the mean maximum log-probability across valid CTC output frames. Formally, for a sequence of length T, the acoustic model uncertainty is defined as:

$$U_{\text{AM}} = -\frac{1}{T}\sum_{t=1}^{T}\max_{c}\log p(c \mid t) \tag{1}$$

where higher values indicate lower confidence in the predicted phoneme sequence. This metric is computed directly from the CTC output without any additional inference overhead.

Prior to adopting the sequence-score approach, we evaluated Monte Carlo Dropout (MC-Dropout) as an alternative uncertainty estimator. By keeping dropout layers active at inference time and running $N = 20$ stochastic forward passes, we computed the predictive entropy of the mean output distribution as the uncertainty proxy. However, this approach yielded a Pearson correlation of only 0.31 with PER on the validation set — substantially weaker than the sequence-score method. We attribute this to two factors: (1) the DB-Conformer backbone contains Batch Normalisation layers in the temporal stem, which behave differently under `model.train()` than at inference time; and (2) the CTC objective already produces peaked output distributions at the frame level, leaving limited variance for dropout to exploit. Given these limitations, we adopted the sequence-score method described above.

Evaluated on the full validation set of 1,378 trials, the acoustic model achieves a mean PER of 12.41% (median 9.09%). The uncertainty score demonstrates a Pearson correlation of 0.622 with PER, confirming that the score is a meaningful predictor of decoding quality.

Stratifying the validation set into four equal-sized quartile bins reveals a clear monotonic relationship between uncertainty and error rate, as shown in Table 10.

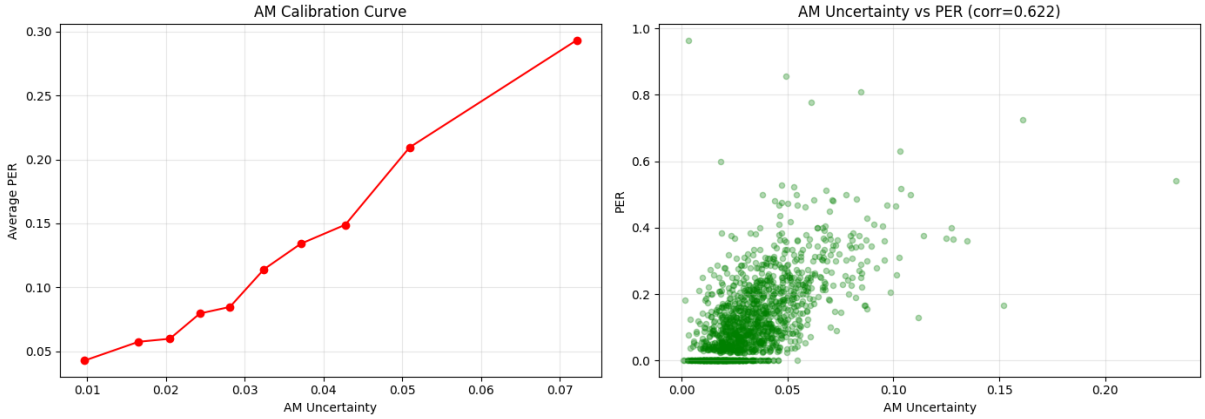

Figure 5: Acoustic Model Calibration Curve and Uncertainty vs PER

Table 10: Acoustic Model Uncertainty Stratification

Confidence Level	Samples	Mean PER	Median PER
HIGH	345 (25.0%)	5.05%	2.56%
MEDIUM	344 (25.0%)	7.93%	6.35%
LOW	344 (25.0%)	12.66%	11.11%
VERY LOW	345 (25.0%)	23.98%	22.58%

Samples in the lowest uncertainty quartile (HIGH confidence) achieve a mean PER of only 5.05%, compared to 23.98% in the highest uncertainty quartile (VERY LOW confidence) — a nearly five-fold difference. This gradient suggests that the acoustic model's confidence is well-calibrated with respect to its actual phoneme-level accuracy.

5.2 Language Model Uncertainty

5.2.1 BART

For the BART-large model, sequence-level uncertainty is derived from the beam search score returned directly by the `generate()` function. Specifically, we use the negated length-normalised log-probability of the best beam as the uncertainty estimate:

$$U_{\text{LM}}^{\text{BART}} = -\text{score}_{\text{beam}} \tag{2}$$

On the validation set, BART achieves a mean WER of 27.84% (median 20.00%). The LM uncertainty score shows a Pearson correlation of 0.544 with WER, and 0.378 with the

upstream acoustic PER. The AM uncertainty retains a correlation of 0.476 with WER, indicating that both sources of uncertainty contribute complementary information.

The confidence stratification in Table 11 reveals a pronounced spread across tiers. However, 43.8% of samples fall into the VERY LOW confidence tier with a mean WER of 46.66%, suggesting that BART's beam scores are not always tightly calibrated.

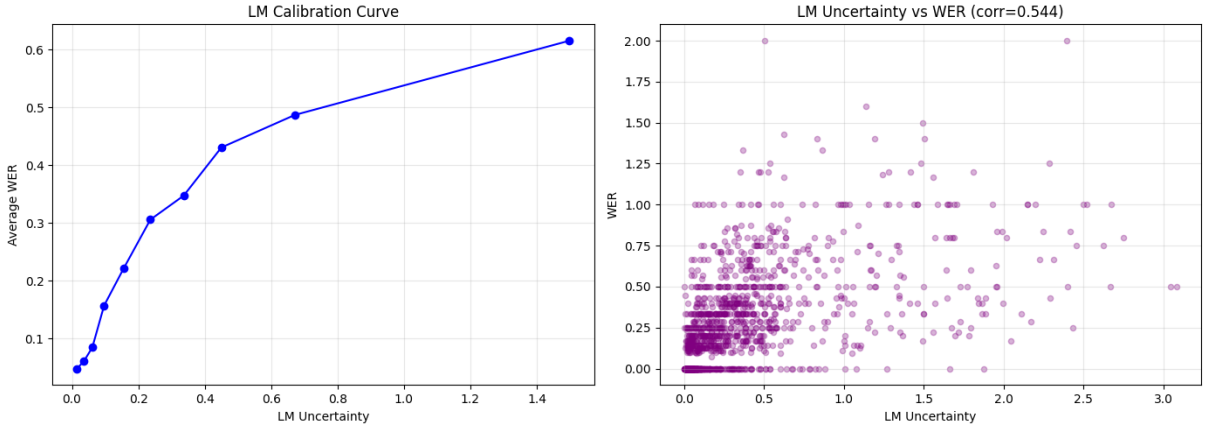

Figure 6: BART Pipeline Uncertainty Analysis

Table 11: BART LM Uncertainty Stratification

Confidence Level	Uncertainty Range	Samples	Mean WER	Median WER
HIGH	< 0.05	285 (20.7%)	5.46%	0.00%
MEDIUM	0.05–0.15	320 (23.2%)	14.01%	0.00%
LOW	0.15–0.25	170 (12.3%)	24.60%	22.22%
VERY LOW	> 0.25	603 (43.8%)	46.66%	40.00%

5.2.2 Qwen 2.5-7B

For the Qwen 2.5-7B model, which follows a decoder-only causal language modelling architecture, beam search scores are not directly accessible via `sequences_scores`. Instead, we compute uncertainty using `compute_transition_scores()`, which returns the per-token log-probability under the normalised output distribution. The sequence-level uncertainty is:

$$U_{\text{LM}}^{\text{Qwen}} = -\frac{1}{|y|} \sum_{i=1}^{|y|} \log p(y_i \mid y_{<i}, x) \tag{3}$$

where y denotes the generated token sequence and x the input phoneme prompt. This is mathematically equivalent to the mean negative log-likelihood of the generated text, and provides a principled measure of the model's token-level confidence.

Qwen achieves a substantially lower mean WER of 22.35% (median 12.50%) compared to BART, representing a relative improvement of approximately 20%. The LM uncertainty

demonstrates a notably stronger correlation with WER at 0.809, compared to 0.544 for BART. The correlation between LM uncertainty and acoustic PER also reaches 0.727.

The confidence stratification in Table 12 shows a particularly sharp separation between tiers. Samples in the HIGH confidence quartile achieve a near-perfect mean WER of 0.65%, while VERY LOW confidence samples reach 57.04%. This 88-fold difference substantially exceeds what is observed for BART (approximately 8.5-fold).

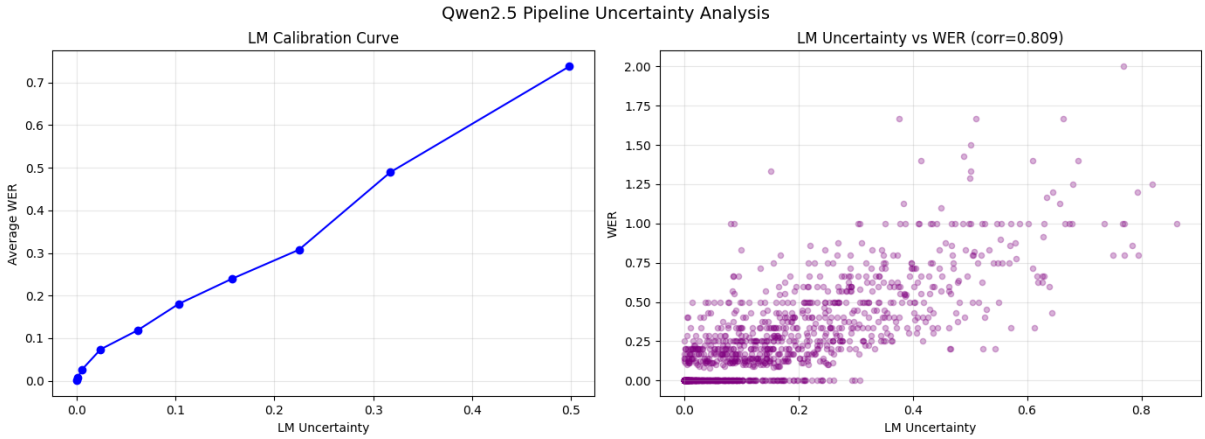

Figure 7: Qwen 2.5 Pipeline Uncertainty Analysis

Table 12: Qwen 2.5 LM Uncertainty Stratification

Confidence Level	Uncertainty Range	Samples	Mean WER	Median WER
HIGH	< 0.005	345 (25.0%)	0.65%	0.00%
MEDIUM	0.005–0.084	344 (25.0%)	8.57%	0.00%
LOW	0.084–0.227	344 (25.0%)	23.12%	20.00%
VERY LOW	> 0.227	345 (25.0%)	57.04%	50.00%

5.3 Comparative Analysis

Table 13 summarises the uncertainty–error correlations across both pipelines. The AM uncertainty correlation with PER is consistent across both models (0.637), as both use the same acoustic model. The key differentiator is the LM uncertainty quality: Qwen's token-level mean log-probability provides a substantially stronger signal ($r = 0.809$) than BART's beam score ($r = 0.544$).

Table 13: Uncertainty–Error Rate Correlations

Metric	BART Pipeline	Qwen Pipeline
AM Uncertainty vs PER	0.637	0.637
AM Uncertainty vs WER	0.476	0.494
LM Uncertainty vs WER	0.544	0.809
LM Uncertainty vs PER	0.378	0.727
Mean WER	27.84%	22.35%
Mean PER (AM)	12.35%	12.49%

These results suggest that the per-token log-probability computed via `compute_transition_scores` is a more reliable uncertainty proxy than beam-level scores for this task. In a deployment scenario, the Qwen uncertainty score could be used to selectively flag low-confidence predictions for human review, with the HIGH confidence quartile (25% of samples) achieving near-perfect transcription at 0.65% WER.

6 Conclusion and Future Work

6.1 Conclusion

This research successfully demonstrates a robust, two-stage neural decoding pipeline capable of translating continuous, 512-channel electrophysiological brain signals into highly accurate English text.

The first stage, the Acoustic Model (AM), utilizes a heavily adapted DBConformer architecture to decode neural signals into 41 distinct phoneme classes. By implementing structural innovations like overlapping temporal patches and a dedicated Connectionist Temporal Classification (CTC) head, the largest DBConformer variant (512 embedding dimension) achieved a mean Phoneme Error Rate (PER) of 12.41%.

The second stage, the Language Model (LM), effectively functions as an error-correcting translation engine. By fine-tuning a Qwen 2.5-7B model using QLORA on a strategic mix of real and synthetically noised phoneme sequences, the pipeline achieved a mean Word Error Rate (WER) of 22.35% and a ROUGE-L of 0.804. Notably, the model demonstrated a strong capacity to correct flawed acoustic predictions, frequently outputting perfect text (WER of 0) even when the input phoneme sequence contained errors.

A standout achievement of this pipeline is its highly calibrated uncertainty estimation. The researchers discovered that evaluating the LM's per-token mean log-probability yielded a remarkably strong correlation (r=0.809) with the actual WER. This allows the system to accurately stratify predictions by confidence; in fact, the highest-confidence quartile of samples achieved a near-perfect mean WER of just 0.65%.

6.2 Future Work

While the current pipeline yields strong metrics, several avenues remain for optimizing accuracy and preparing the system for real-world clinical use:

- **Acoustic Architectural Scaling:** The DBConformer's performance improved consistently as embedding dimensions scaled up, indicating the model's capacity has not yet saturated. Future iterations should explore a learned gating mechanism to intelligently reintegrate the bypassed spatial branch, capturing complex inter-channel relationships without compromising necessary temporal alignment.
- **Mitigating Hallucination:** To address the specific LLM failure modes identified (such as semantic divergence and catastrophic hallucination) , future models should employ constrained decoding techniques. Implementing a lexicon trie during beam search could force the model to strictly output valid dictionary words, while stricter prompting rules could minimize contraction mismatches.
- **Deployment Workflows:** The excellent calibration of the Qwen uncertainty scores provides a blueprint for immediate practical deployment. Systems can be designed

to automatically accept high-confidence text generations while routing the lowest-confidence quartile to a human operator for review.

- **End-to-End Clinical Applicability:** To achieve true clinical viability, future research must address cross-subject generalization. Furthermore, developing a joint, end-to-end training paradigm where the acoustic model's uncertainty scores are passed as soft evidence directly into the language model could yield a more unified and resilient decoding system.

References

[1] Y. Song et al., "EEGConformer: Convolutional Transformer for EEG Decoding and Visualization," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 2023.

[2] X. Wang et al., "DBConformer: Dual-Branch Convolutional Transformer for EEG Decoding," 2025.

[3] D. Card et al., "An Accurate and Rapidly Calibrating Speech Neuroprosthesis," *New England Journal of Medicine*, 2024.

Appendix: Phoneme Confusion Map

Listing 1: Phoneme confusion map used for synthetic data augmentation

```
phoneme_confusion_map = {

    # Plosives/Stops (Confused by voicing: unvoiced vs. voiced)
    27: [7, 31, 20],    # P (27) -> B, T, K
    7:  [27, 9, 15],    # B (7)  -> P, D, G
    31: [9, 27, 20],    # T (31) -> D, P, K
    9:  [31, 7, 15],    # D (9)  -> T, B, G
    20: [15, 31, 27],   # K (20) -> G, T, P
    15: [20, 9, 7],     # G (15) -> K, D, B

    # Fricatives (Confused by voicing and similar high-frequency
       hiss)
    14: [35, 32, 29],   # F (14)  -> V, TH, S
    35: [14, 10, 38],   # V (35)  -> F, DH, Z
    32: [10, 14, 29],   # TH (32) -> DH, F, S
    10: [32, 35, 38],   # DH (10) -> TH, V, Z
    29: [38, 30, 32],   # S (29)  -> Z, SH, TH
    38: [29, 39, 10],   # Z (38)  -> S, ZH, DH
    30: [39, 29, 8],    # SH (30) -> ZH, S, CH
    39: [30, 38, 19],   # ZH (39) -> SH, Z, JH
    16: [32, 14],       # HH (16) -> TH, F

    # Affricates (Stop + Fricative combos)
    8:  [19, 30, 31],   # CH (8)  -> JH, SH, T
    19: [8, 39, 9],     # JH (19) -> CH, ZH, D

    # Nasals (Acoustically very similar)
    22: [23, 24],       # M (22)  -> N, NG
    23: [22, 24],       # N (23)  -> M, NG
    24: [23, 22],       # NG (24) -> N, M

    # Liquids and Glides (Often blend into vowels)
    21: [28, 36, 37],   # L (21)  -> R, W, Y
    28: [21, 36],       # R (28)  -> L, W
    36: [21, 28, 34],   # W (36)  -> L, R, UW
    37: [21, 18],       # Y (37)  -> L, IY
```

```
    # === VOWELS ===

    # Front Vowels
    18: [17, 13],       # IY (18) -> IH, EY
    17: [18, 11],       # IH (17) -> IY, EH
    11: [17, 2],        # EH (11) -> IH, AE
    2:  [11, 1],        # AE (2)  -> EH, AA

    # Back / Central Vowels
    1:  [4, 3],         # AA (1)  -> AO, AH
    4:  [1, 33],        # AO (4)  -> AA, UH
    33: [34, 4],        # UH (33) -> UW, AO
    34: [33, 25],       # UW (34) -> UH, OW
    3:  [11, 33, 12],   # AH (3)  -> EH, UH, ER
    12: [3, 28],        # ER (12) -> AH, R

    # Diphthongs (Gliding vowels)
    13: [11, 18, 6],    # EY (13) -> EH, IY, AY
    6:  [1, 18, 13],    # AY (6)  -> AA, IY, EY
    26: [4, 18],        # OY (26) -> AO, IY
    5:  [1, 34, 25],    # AW (5)  -> AA, UW, OW
    25: [4, 34, 5]      # OW (25) -> AO, UW, AW
}
```